

Java Shared Memory Performance Races: Implementation and Testing of Synchronized, Unsynchronized, and Null Classes

By: Ivan Tran

1 Introduction

Using the UnsafeMemory class, I ran the built-in main method with a shell command like this: “time timeout 3600 java UnsafeMemory Platform 5 Synchronized 8 100000000”. This specific command ran the main method of the UnsafeMemory class with the arguments Platform, 5, Synchronized, 8, and 1000000000. Platform refers to using platform threads, or Virtual would mean using the alternative Virtual threads. 5 says to use a state array of 5 entries; the other value used was 100. Synchronized refers to the swap transitions either being synchronized, unsynchronized, or null. So, the usable arguments for the shell command were “Synchronized”, “Unsynchronized”, and “Null”. 8 refers to the number of threads; the number of threads varied from 1, 8, and 40. The final value of 1000000000 (100 million) represents the total number of swap transitions we performed and this value was not changed throughout running the test harness.

Furthermore, a note on how to read the table: “P5S1” stands for the test results of Platform 5 Synchronized 1.

2 Results

To concisely discuss the data from the test harness, the analysis is split into three sections: using platform threads, using virtual threads, and comparing platform versus virtual threads.

2.1 Platform Thread Analysis for Classes (varying threads and array size)

The null class scores the lowest total real-time while varying all conditions. The synchronized and unsynchronized classes’ total real-time increased as the number of threads increased. With more threads, a higher average real swap time causes a slowdown. For the synchronized class, increasing threads at state array size 5 is an overall increase of 5%, and size 10 is 15% of total real-time. For the unsynchronized class, the increase is 159% (size 5) and 132% (size 10) while causing output sum inaccuracies. For both

classes, holding threads constant, there are minimal differences between size 5 and 10 state arrays.

Conclusively, the unsynchronized class performs similarly to the synchronized class yet suffers from many inaccurate swaps which led to output sum mismatches for all unsynchronized platform tests, excluding single thread tests. From the scale of 100 million swap transitions, there is a small performance uplift when using more threads and a smaller state array size. The worst performer (not including inaccurate test runs) was at 40 threads using size 100 for the synchronized class. The best non-trivial performer (reliable swaps and isn’t Null Class) was at 1 thread using size 5 for the unsynchronized class.

2.2 Virtual Thread Analysis for Classes (varying threads and array size)

Similar to the platform threads, the Null class’s total real times were the lowest again. The synchronized and unsynchronized classes’ total real-time increased as the thread count increased. Contrary to using Platform threads, the increase in total time on virtual threads was non-negligible at 100% for unsynchronized and over 100% for sync. Increasing threads at larger array sizes caused larger total times. Despite having lower total real-time, the unsynchronized class has output sum mismatches for all tests except the single-threaded tests.

The unsynchronized class offers more robust performance at the expense of accuracy. When increasing the number of threads on the two different state array sizes, for both synchronized and unsynchronized there are not large differences in how real-times will increase as the number of threads increases. The biggest difference between synchronized and unsynchronized classes is the average real swap time, such that unsynchronized has much lower swap times given that it does not wait for any events before running any instructions.

The worst performer was at size 100, 40 threads, and using the synchronized class. The best non-trivial

performer was at size 5, 1 thread, and using the unsynchronized class.

2.3 Comparing Platform versus Virtual Threads (keeping thread count and array size constant)

When comparing the real-time performance without changing the number of threads or state array size, virtual threads perform significantly worse. For the synchronous class, at low thread counts (1), the difference is negligible but at high thread counts (40), the difference is a 100% increase in real-time (P5S40 vs V5S40). For the unsynchronous class, virtual threads on average are slower than platform threads.

3 Conclusion

Platform threads are generally faster than virtual threads even when varying the state array size and number of threads used. Increasing the state array size generally does not necessarily make performance better but may lead to marginal performance gains or losses. At the number of swap transitions tested, increasing the number of threads leads to an increase in overall real-time (decrease in performance). An observation is that increasing the number of threads increases the swap time used which is causing the increase in overall time. Avoiding the usage of the “synchronize” keyword (making your class unsynchronized) will lead to race conditions that cause output sum mismatch and overall completely inaccurate results.

Appendix

Raw Data:

	P5S1	P5S8	P5S40	P100S1	P100S8	P100S40
Total real time:	3.4984	3.66933	3.67044	3.48189	3.55817	4.00325
Average real swap time (ns):	34.9894	293.546	468.18	34.8189	284.654	1601.3
Output sum Mismatch:	N/A	N/A	N/A	N/A	N/A	N/A
Real (s) :	3.653	3.782	3.784	3.597	3.679	4.101
User (s) :	3.592	4.594	4.683	3.553	4.408	5.626
Sys (s) :	0.038	0.173	0.204	0.045	0.222	0.31
	P5N1	P5N8	P5N40	P100N1	P100N8	P100N40
Total real time:	1.37735	0.402763	0.415293	1.37958	0.407089	0.446609
Average real swap time (ns):	13.7735	32.221	166.117	13.7958	32.5671	178.644
Output sum Mismatch:	N/A	N/A	N/A	N/A	N/A	N/A
Real (s) :	1.49	0.507	0.512	1.49	0.507	0.547
User (s) :	1.466	1.643	1.627	1.471	1.611	1.767
Sys (s) :	0.038	0.035	0.039	0.033	0.045	0.048
	P5U1	P5U8	P5U40	P100U1	P100U8	P100U40
Total real time:	1.3842	3.629	3.5973	1.52406	3.50384	3.55058
Average real swap time (ns):	13.842	290.32	1438.92	15.2406	280.308	1420.23
Output sum Mismatch:	N/A	-1174 != 0	-4702 != 0	N/A	-18689 != 0	-17724 != 0
Real (s) :	1.483	3.728	3.694	1.627	3.613	3.649
User (s) :	1.469	14.272	13.675	1.627	13.754	13.987
Sys (s) :	0.034	0.06	0.108	0.03	0.07	0.076
	V5S1	V5S8	V5S40	V100S1	V100S8	V100S40
Total real time:	3.52452	8.95494	7.33381	3.51324	7.11349	7.48486
Average real swap time (ns):	35.2452	716.395	2933.52	35.1324	569.079	2993.95
Output sum Mismatch:	N/A	N/A	N/A	N/A	N/A	N/A
Real (s) :	3.64	9.08	7.45	3.755	7.218	7.603
User (s) :	3.536	17.288	9.588	3.555	14.096	14.841
Sys (s) :	0.07	2.992	1.94	0.05	2.377	2.688
	V5N1	V5N8	V5N40	V100N1	V100N8	V100N40
Total real time:	1.26112	0.457621	0.367206	1.36824	0.436048	0.463717
Average real swap time (ns):	12.6112	36.6097	146.882	13.6824	34.8838	185.487
Output sum Mismatch:	N/A	N/A	N/A	N/A	N/A	N/A
Real (s) :	1.391	0.571	0.471	1.474	0.54	0.569

User (s) :	1.38	1.862	1.52	1.467	1.72	1.86
Sys (s) :	0.043	0.042	0.041	0.037	0.035	0.032
	V5U1	V5U8	V5U40	V100U1	V100U8	V100U40
Total real time:	1.32886	2.48198	2.63913	1.51249	3.75815	3.86133
Average real swap time (ns):	13.2886	198.559	1055.65	15.1249	300.652	1544.53
Output sum Mismatch:	N/A	-26300 != 0	-17995 != 0	N/A	-25183 != 0	-31134 != 0
Real (s) :	1.479	2.595	2.748	1.619	3.884	3.969
User (s) :	1.421	9.315	10.155	1.594	14.547	15.192
Sys (s) :	0.041	0.078	0.08	0.042	0.084	0.062